Maintainr Installation and Professional Publishing Guide

Version: August 2026
Audience: Maintainr project owner, developer, or deployment administrator
Purpose: Configure, test, and publish Maintainr as a professional multi-tenant property-maintenance SaaS.

> **PostgreSQL-first private target:** This release uses the standalone PostgreSQL schema in `POSTGRESQL_SCHEMA.sql` and self-hosted email/password authentication. Storage and scheduled-callback adapters remain deployment-specific and must be configured for the independent host you choose. Purchase-code activation is not required for your private installation.

1. What you must create

For the first professional release, create the accounts in the following table. You do not need every optional account on day one.

Account or service	Required?	What it provides	Where to create it
PostgreSQL provider	Yes	Database for organizations, users, sessions, tickets, reminders, and audit logs	Any PostgreSQL provider or your own server
Domain registrar/DNS provider	Recommended	Professional website address such as `app.example.com`	Any domain provider you trust
Resend	Recommended for email	Ticket, assignment, status, and maintenance-reminder email	resend.com/signup
Twilio	Optional	SMS reminders and SMS ticket notifications	twilio.com/try-twilio

Do not upload the PostgreSQL SQL file into Firebase. Firebase is not a PostgreSQL database. Import `POSTGRESQL_SCHEMA.sql` into a PostgreSQL service such as your own

VPS, managed PostgreSQL provider, or cloud database. The application must use the matching `DATABASE_URL`.

2. Configure the Maintainr project

Open the Maintainr project Management UI. In **Settings → General**, configure the website name, visibility, domain, and favicon as available. In the developer settings inside the application, configure the organization's project name, Arabic project name, logo URL, primary color, accent color, and notification channel toggles.

Use the exact roles already implemented in the application: `PROPERTY_MANAGER`, `TENANT`, `TECHNICIAN`, and `FLAT_OWNER`. The manager portal is `/manager`, the tenant portal is `/tenant`, the technician portal is `/technician`, and the flat-owner portal is `/owner`. After authentication, Maintainr routes the user to the portal matching the stored role.

Create a first test organization with at least one property and several units. Create or invite one account for each role. Generate six-digit unit access codes from the manager portal and verify that a tenant can use `/join-unit` to bind to the correct unit.

3. Email setup with Resend

Resend requires an account, a verified sending domain, and an API key before application email can be sent.[1] Create an account at resend.com/signup, then open the Resend dashboard.

3.1 Verify your sending domain

In Resend, open **Domains**, choose **Add Domain**, and enter a domain that you own, such as `example.com` or `mail.example.com`. Resend will display DNS records. Open your domain registrar's DNS manager and add the records exactly as Resend shows them. Return to Resend and wait for the domain to become verified. Use an address such as `notifications@example.com` as the sender after verification.[2]

3.2 Create the API key

In Resend, open **API Keys**, choose **Create API Key**, give it a name such as `maintainr-production`, select the minimum permission required for sending, and copy the key immediately. Resend API keys are secret tokens used to authenticate API requests.[3]

3.3 Add the email values to Maintainr

Use the project's managed **Settings → Secrets** area or the secure secret-entry card. Add the following values. Never paste these into React code, never commit them to Git, and never put them into the Developer Settings form.

Environment variable	Value to enter	Required
`RESEND_API_KEY`	The Resend API key beginning with the provider's key prefix	Yes for email delivery
`NOTIFICATION_FROM_EMAIL`	The verified sender, for example `notifications@example.com`	Yes for email delivery

Keep the application's email channel disabled until both values are configured and a test email has arrived successfully. Then enable email from the Developer Settings screen.

4. Optional SMS setup with Twilio

Twilio SMS is optional. Email should be enabled and tested first. Twilio's SMS quickstart requires a Twilio account, an SMS-capable number, and the account credentials used to authenticate API requests.[4]

Create an account at twilio.com/try-twilio and open the Twilio Console. The Console dashboard displays the **Account SID**. It is normally an identifier beginning with `AC`. The **Auth Token** is available in the account security area; treat it as a password and do not share it.[5]

Next, obtain a phone number with SMS capability. In the Twilio Console, use **Phone Numbers → Buy a number** or the equivalent number setup flow, select a number that supports SMS in your target country, and copy the number in international E.164 format. Twilio's quickstart describes obtaining a number from the Account Dashboard and using it as the sender.[4]

Add these values as managed secrets. Leave `TWILIO_ENABLED=false` until all required values are present and tested.

Environment variable	Value to enter	Required
`TWILIO_ENABLED`	`true` only after configuration and testing; otherwise `false`	Optional
`TWILIO_ACCOUNT_SID`	Twilio Console Account SID	Required only for SMS
`TWILIO_AUTH_TOKEN`	Twilio Console Auth Token	Required only for SMS
`TWILIO_FROM`	SMS-capable Twilio phone number in E.164 format	Required only for SMS

Trial accounts can have destination restrictions, verification requirements, or account limitations. Confirm the Twilio Console's current requirements for the countries where your users live before enabling SMS.

5. PostgreSQL database setup

Create an empty PostgreSQL database under your own account. Download `POSTGRESQL_SCHEMA.sql` from this project and run it once with a PostgreSQL client such as `psql`, your provider's SQL console, or a database administration tool. The script creates the enums, tables, indexes, foreign keys, validation checks, and updated-at triggers required by Maintainr.

From a terminal, the standard import command is:

```
psql "$DATABASE_URL" -v ON_ERROR_STOP=1 -f POSTGRESQL_SCHEMA.sql
```

If your provider gives separate host, port, database, user, and password fields, create a connection string in this form and then run the same command:

```
postgresql://DB_USER:DB_PASSWORD@DB_HOST:5432/DB_NAME?sslmode=require
```

After the import, verify that these tables exist: `organizations`, `properties`, `units`, `users`, `sessions`, `passwordResetTokens`, `tickets`, `ticketMedia`, `ticketLogs`,

`maintenanceReminders`, `reminderRuns`, `reminderAcknowledgements`, and `developerSettings`. The SQL file is designed to be safely rerun for an empty or partially initialized database.

After importing the schema, set `DATABASE_URL` to a PostgreSQL connection string. Use SSL in production, create a least-privilege application database user, and configure automated backups before importing real organizations or tickets. Do not run the schema against the current managed project database because that database is not the user's independent PostgreSQL target.

The PostgreSQL schema is server-side configuration. Never place `DATABASE_URL` in client code or a `VITE_` variable.

Environment variable	Purpose
`DATABASE_URL`	PostgreSQL connection string for the server
`BOOTSTRAP_MANAGER_EMAIL`	Email that receives the first `PROPERTY_MANAGER` role during signup
`AUTH_BASE_URL`	Public HTTPS URL used in password-reset links

Do not commit database passwords, authentication secrets, provider keys, or service-account files. Keep PostgreSQL as the only source of truth for organizations, users, tickets, media metadata, reminders, and audit logs.

6. Where environment values belong

Use the project's secure environment/secrets management interface for real credentials. The repository contains `.env.production.example` and `.env.notifications.example` as reference templates only. They contain blank values and must remain safe to commit.

Value type	Correct location
Resend API key, Twilio Auth Token, database credentials, bootstrap email, and reset configuration	Managed project secrets
Project name, Arabic project name, logo URL, primary color, accent color	Maintainr Developer Settings
Email/SMS enabled or disabled	Maintainr Developer Settings, with safe disabled defaults
PostgreSQL connection and independent auth secrets	Server-side deployment secret manager only
Public domain and favicon	Project Management UI settings

If you are asked to enter a secret in chat, do not paste it into the conversation. Use the secure secret card or project secret manager instead.

7. Test the complete product before publishing

First, set `BOOTSTRAP_MANAGER_EMAIL` and test local email/password sign-in and sign-up with separate accounts. Public Tenant and Technician access is requested through `/apply`; the application is saved in the Manager dashboard. With Resend configured, Maintainr emails both the Property Manager and applicant when the application is submitted. The Manager approves or rejects it, and an approved applicant receives a bilingual, single-use invitation link to create a personal password; no permanent password is sent by email. Test `/forgot-password` and `/reset-password` with Resend configured, then verify logout revokes the active session. Confirm that a new user can complete `/join-unit` with a valid six-digit code and that an invalid code is rejected. Confirm that each exact role reaches only its own portal.

Next, use the manager portal to create a ticket, assign a technician, change priority, and review the audit history. Use the tenant portal to create a request with multiple attachments. Use the technician portal to upload proof media and resolution notes; verify that resolution cannot be submitted without both. Use the owner portal to review the scoped reminder and ticket information.

Create one one-time reminder and one recurring reminder. Confirm that the manager can see and edit them, that tenants/technicians/owners see only their scoped reminders, and that acknowledgement state is visible. Keep email and SMS disabled for the first local test, then enable email and send one production test. Enable SMS only after its provider test succeeds.

Switch to Arabic and verify the public pages, authentication, onboarding, manager dashboard, reminder form, reminder inbox, ticket forms, and mobile layouts. Verify that the URL query preview `?lang=ar` does not change the user's persisted language preference.

8. Deploy and operate independently

Before deployment, confirm that all production secrets are configured, the notification sender domain is verified, the PostgreSQL schema has been imported, the latest tests pass, and a backup has been taken. For Vercel or Netlify, deploy the frontend and adapt the Node/tRPC server to the provider's server-function model; recurring reminders need a separate cron or worker service. For cPanel, use the Node.js Application Manager or Passenger, configure the server environment values, enable SSL, and configure a reliable cron job. The application must not depend on a Manus login redirect for private operation.

After deployment, open the production URL in a private browser window and test sign-in, sign-up, password reset, logout, `/join-unit`, each portal URL, media upload, email, reminders, and the language switch. Review production logs after the first scheduled reminder callback. Keep database backups enabled and document who can rotate secrets, recover the organization owner account, and disable notification channels during an incident.

References